



Machine Learning for Petroleum Engineers Using Python

Módulo 3 · Clase 3

No Linealidad: Árboles de Decisión y Random Forest



Carlos Enrique Mosquera Trujillo



SLB Ecuador

El Problema: la Realidad no Está Obligada a Ser una Recta

Hasta ahora usamos modelos lineales. Funcionan muy bien cuando el efecto es parejo.
¿Pero qué pasa cuando no lo es?

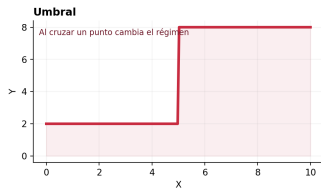
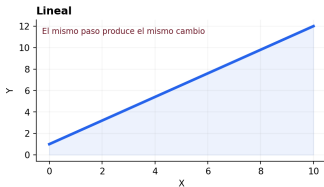
- 🔧 Abrir el choke 10 puntos no siempre agrega el mismo caudal.
- 📈 La producción cae rápido al inicio y lentamente después.
- 🌡 Temperatura y viscosidad tienen una relación exponencial.
- 🔧 Una clase puede envolver a otra: no existe una línea que las separe.

La pregunta de la clase

¿Cómo construimos modelos capaces de reconocer **curvas, umbrales e interacciones**, sin tener que escribir de antemano la ecuación física exacta?

*Hoy veremos la misma solución en dos decisiones:
estimar un número y predecir una clase.*

Cuatro Formas que una Recta no Puede Representar Bien



La no linealidad no significa “caos”: significa que el cambio **depende de dónde estamos**, puede saturarse o cambiar de régimen.

El Límite no es Falta de Datos: es la Forma del Modelo

Regresión lineal

Solo puede ajustar un plano:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$$

Cada variable conserva el mismo efecto en todo el rango.

Regresión logística

Convierte ese plano en probabilidad, pero su frontera de decisión sigue siendo **una línea o un plano**.

Cambia la salida; no cambia la geometría interna.

Si el patrón es curva, ninguna cantidad de entrenamiento hará que una recta se doble.

Dónde Estamos

✓ Ya tenemos:

- ✓ Regresión lineal: predecir un **número**
- ✓ Regresión logística: predecir una **etiqueta**
- ✓ Métricas + **costo de negocio** y umbral
- ✓ Train/test, estandarización, binarización

*Pero ambos modelos comparten un límite: por dentro son **una recta**.*

▶▶ Hoy:

- Qué es la **no linealidad** y por qué el mundo real casi siempre es así
- Ver a la recta **fracasar** (las lunas y un problema real)
- **Árboles de decisión**: cutoffs automatizados
- ★ **Random Forest**: la sabiduría de la multitud

*Reto de hoy: un **medidor virtual de flujo**.*

¿Qué es la No Linealidad?

Cuando el paso deja de ser parejo

Recuerden: Lineal = el Paso Parejo

En la Clase 1 lo definimos: lineal es cuando el mismo paso en X produce siempre el mismo paso en Y.

✓ LINEAL (el paso vale igual)

Tanquear el carro: el galón 1 cuesta lo mismo que el galón 10.

Presión hidrostática: cada 100 m suman $\approx$ lo mismo.

✗ NO LINEAL (el paso cambia)

Abrir el choke de 20 % a 30 % **no** da lo mismo que de 70 % a 80 %.

El declive: el primer año el pozo cae muchísimo; el quinto, casi nada.

→ **No lineal** = el efecto de una variable **depende de dónde estás** (y a veces, del valor de **otras** variables).

⚠ Curvas, mesetas, umbrales, interacciones: así se comporta **la mayoría** de la física de producción.

El Mundo Real Rara Vez es una Recta

Piensen en su operación: ¿cuántas de estas relaciones son “parejas”?

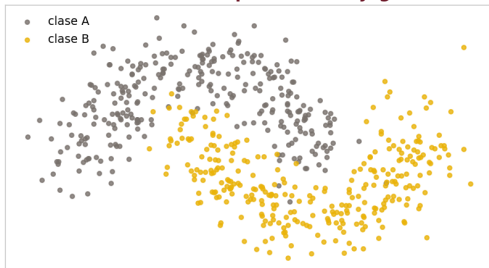
Relación	Por qué NO es lineal
Choke vs caudal	al inicio cada punto de apertura suma mucho; cerca del máximo, casi nada (flujo crítico)
Tiempo vs producción	la curva de declive: cae rápido y luego lento
Frecuencia de bomba vs caudal	sube hasta una meseta : el pozo no da más
Presión vs caudal (IPR)	la curva IPR es...una curva
Temperatura vs viscosidad	exponencial: unos grados cambian todo

💡 La recta es el modelo **más simple y explicable** — por eso empezamos ahí. Pero cuando la física es curva, **necesitamos modelos que doblen**.

Un Experimento: las Dos Lunas

Un problema de juguete, diseñado para ver el límite de la recta.

Las dos lunas: un problema de juguete

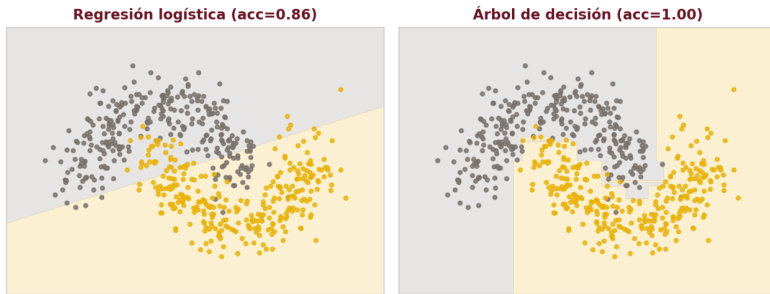


- Dos grupos con forma de **medialunas entrelazadas**.
- 👁 Cualquier persona los separa **a ojo**, sin pensar.
- ? Pregunta: ¿puede separarlos una **línea recta**?

Este dataset (`make_moons`) es el examen clásico de no linealidad en ML.

La Recta Fracasa; el Árbol lo Resuelve

La regresión logística solo puede trazar una frontera recta. Miren lo que pasa.

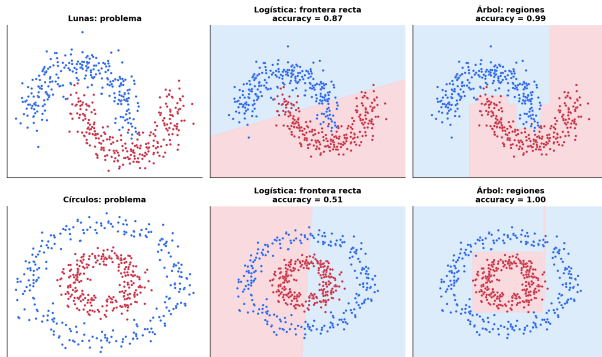


✗ La logística se queda en **0.86**: su recta corta las dos lunas por la mitad, **no puede hacer nada mejor**.

✓ El árbol dibuja una frontera de **escalones** que sigue la forma: **1.00**. Hoy aprenderemos cómo.

No es Solo una Luna: También Puede Haber Círculos

Dos patrones fáciles para una persona, imposibles de separar perfectamente con una sola recta.



El árbol no dibuja una curva suave: combina muchos cortes horizontales y verticales hasta formar **regiones**. Esa es

El Problema de Hoy

Un medidor virtual de flujo

¿Cuánto Está Produciendo Cada Pozo, Ahora?

Parece una pregunta obvia. No lo es.

- 🔧 Medir el caudal de **un** pozo exige desviar su flujo al **separador de prueba**: se hace cada tantos días, **no** a diario.
- 🕒 Lo que **sí** está siempre disponible: los sensores de **presión, temperatura y choke**.
- 💡 **La idea**: un modelo que **estime la producción** desde esos sensores — un *medidor virtual de flujo*.

Esto es un producto real:

los *virtual flow meters* se venden y operan en la industria hoy. Detrás hay exactamente lo que vamos a construir: un modelo entrenado con datos históricos de pruebas de pozo.

Es **regresión** (predecimos un número), pero hoy la física no nos va a dejar usar una recta.

El Dataset: Operación Diaria de Volve

5 pozos productores de Volve, 7 862 días de operación con sus sensores y su producción medida.

Python

```
op = pd.read_csv(url)
op.shape
```

>_ Output

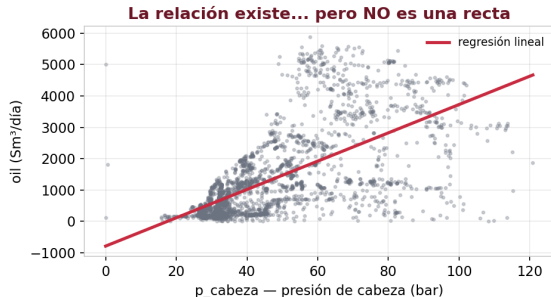
```
(7862, 9)
```

- El mismo campo Volve de las clases anteriores, ahora sus datos **de operación**.
- El target `oil` viene de las **mediciones reales** de producción diaria.

Columna	Qué mide	Dónde se mide
p_fondo	presión en el fondo del pozo (bar)	sensor de fondo
p_cabeza	presión en la cabeza del pozo (bar)	superficie
t_cabeza	temperatura en cabeza (°C)	superficie
choke	apertura de la válvula choke (%)	superficie
dp_choke	caída de presión a través del choke	superficie
horas	horas en operación ese día	registro
oil	el target: producción (Sm ³ /día)	separador

Explorar: la Relación Más Fuerte... es Curva

La correlación más alta con oil es p_cabeza (0.64). Dibujémosla.



- Hay relación, clarísima. Pero la nube tiene **forma de codo**, no de recta.
- ✗ La recta pasa **por en medio de nada**: subestima abajo, sobreestima arriba.
- ⚠ Y esto es **una sola** variable: las interacciones entre presiones y choke lo complican más.

La Regresión Lineal se Queda Corta

Entrenamos la lineal de la Clase 1 con las 6 variables. El resultado:

Python

```
lineal = LinearRegression()
lineal.fit(X_tr, y_tr)

r2 = lineal.score(X_te, y_te)
print("R2:", r2)
```

>_ Output

```
R2: 0.595
MAE: 583 Sm3/dia
```

- ✗ Explica el **60 %** y se equivoca **583 Sm³/día** en promedio — casi **la mitad** de la producción media (1264).
- ➔ No es que el problema sea impredecible: es que **la recta no puede doblar**.
- 💡 Necesitamos un modelo que capture **curvas y umbrales**.

Árboles de Decisión

Cutoffs automatizados

Ustedes Ya Piensan en Árboles

Un cutoff es una decisión con umbral. Los usan todos los días.

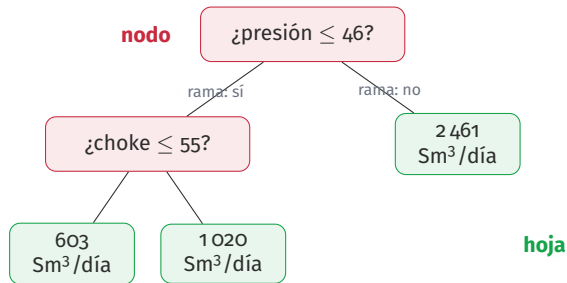
- 🔗 “Si el GR $< 60 \rightarrow$ arena; si no $\rightarrow$ arcilla.”
- 🔗 “Si la presión anular $> X \rightarrow$ revisar; si no $\rightarrow$ seguir.”
- 🔗 Y los encadenan: “si es arena **y** la porosidad $> 15\%$ **y** la resistividad es alta $\rightarrow$ zona de interés.”

Un árbol de decisión es exactamente eso:

una **cascada de cutoffs**... con una diferencia: **la máquina elige los cortes** — qué variable, en qué valor y en qué orden — probando miles de opciones y quedándose con las que **mejor separan** los datos.

No hay fórmula ni coeficientes: solo preguntas de sí/no encadenadas.

Anatomía de un Árbol: Cuatro Palabras

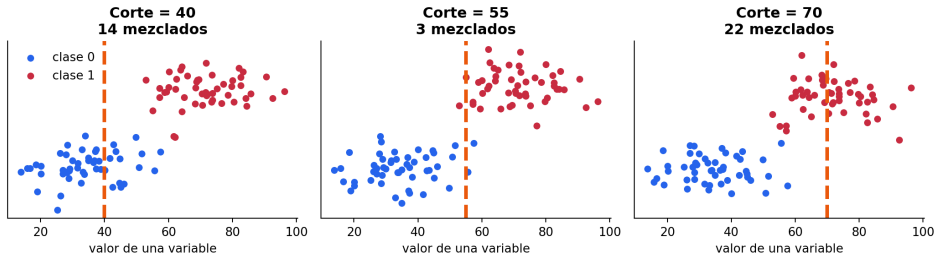


- **Raíz:** la primera pregunta.
- **Nodo:** una pregunta intermedia.
- **Rama:** la respuesta sí/no.
- **Hoja:** la predicción final.

Leer un árbol es seguir una ruta, no resolver una ecuación.

¿Cómo Decide la Primera Pregunta?

Prueba muchos candidatos y conserva el corte que deja grupos más homogéneos.



Para cada variable prueba distintos valores. En clasificación busca grupos con **menos mezcla**; en regresión, grupos con valores **menos dispersos**.

La Misma Mecánica, Dos Tipos de Predicción

Árbol de regresión

Llegan a una hoja estos caudales:

580, 620, 610, 590

Predicción = **promedio**

Predicción final: **600 Sm³/día.**

Árbol de clasificación

Llegan a una hoja 8 arenas y 2 lutitas.

Clase = **voto mayoritario**: arena

Probabilidad = **8/10 = 0.80**

Cambia lo que ocurre en la hoja; la secuencia de preguntas es la misma.

¿Cuándo Deja de Preguntar?

Si no ponemos límites, el árbol puede crear una hoja para casi cada ejemplo.

max_depth

Máximo número de niveles. Controla cuánto puede complicarse.

min_samples_leaf

Mínimo de ejemplos que debe contener una hoja.

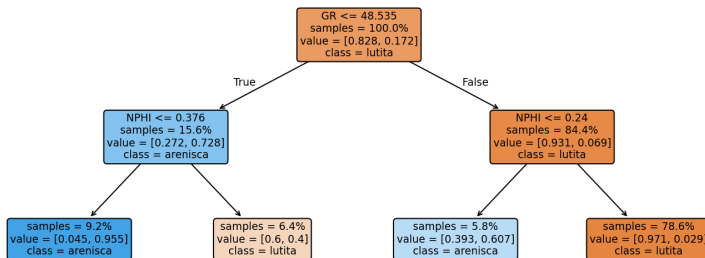
min_samples_split

Mínimo de ejemplos para permitir una nueva pregunta.

No buscamos el árbol que mejor recuerde el entrenamiento. Buscamos el más simple que conserve buen desempeño en **datos nuevos**.

Nuestro Primer Árbol, Dibujado

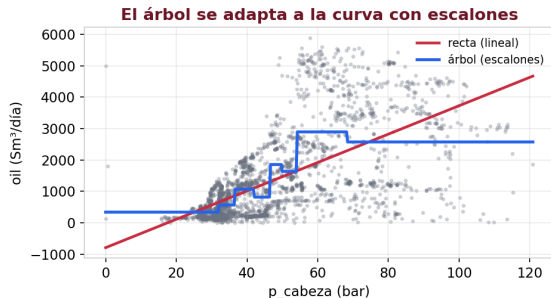
Un árbol de 2 niveles entrenado con nuestros datos. Se lee de arriba hacia abajo.



- Primera pregunta: ¿ $p_cabeza \leq 46$? Si sí, el pozo produce poco (~ 603); si no, mucho (~ 2461).
- ✓ Con solo **3 preguntas**, este árbol ya da $R^2 = 0.66$: **le gana a la regresión lineal con sus 6 variables.**

Por Qué el Árbol Captura lo No Lineal

Cada corte crea un “escalón”. Muchos escalones siguen cualquier curva.



- La **recta** está obligada a mantener la misma pendiente en todas partes.
- El **árbol** predice un valor distinto en cada tramo: donde la curva sube rápido, pone escalones altos.
- ✓ Sin decirle nada de la física: los cortes salen **de los datos**.

En Código: el Mismo Patrón de Siempre

Cambia el modelo, no el flujo. Y un detalle cómodo: los árboles no necesitan estandarización.

Python

```
from sklearn.tree import  
    DecisionTreeRegressor  
  
arbol = DecisionTreeRegressor(  
    max_depth=5, random_state=42)  
arbol.fit(X_tr, y_tr)  
  
print(arbol.score(X_te, y_te))
```

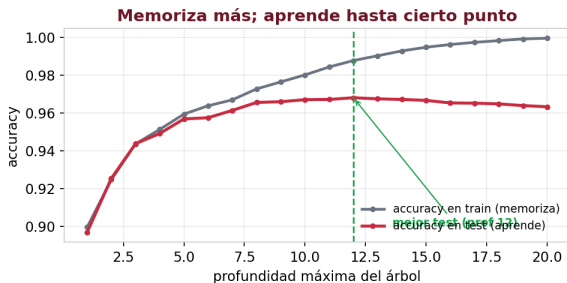
>_ Output

0.886

- ✓ De 0.60 a **0.89** con 5 niveles de preguntas.
- max_depth: cuántos niveles de preguntas puede hacer. Es **la perilla clave**.
- Los cortes comparan cada variable **consigo misma** → las escalas no importan.

La Tentación: ¿y si lo Dejamos Crecer Sin Límite?

Más profundidad = más poder. Hasta que empieza a memorizar.



- Sin límite, el árbol memoriza **cada día** del historial: R^2 **train** = 1.000.
- ⚠ Pero en **test** no mejora más: aprendió el **ruido**, no el patrón.
- ✓ Es el **overfitting** de la Clase 1, ahora **visible** en una curva.

💡 El árbol solo es **poderoso pero indisciplinado**. La solución no es un árbol mejor... son **muchos árboles**.

Random Forest

La sabiduría de la multitud

La Idea: Preguntarle a 200 Ingenieros Distintos

Un experto se equivoca. El promedio de muchos expertos independientes se equivoca mucho menos.

- 🌲 **Random Forest** entrena **cientos de árboles**, cada uno un poco distinto.
- ✂️ ¿Distintos cómo? Cada árbol ve una **muestra distinta de los días** y considera **variables distintas** en cada corte.
- 👥 La predicción final: el **promedio** de todos (o el voto, si es clasificación).

Por qué funciona:

cada árbol memoriza **ruido distinto**. Al promediar, los errores individuales **se cancelan entre sí** y queda el patrón que todos vieron: **la señal**.

Como promediar las estimaciones de 200 ingenieros que estudiaron el campo con informes diferentes.

Random Forest en Código

Una línea distinta. El resto del flujo, idéntico.

Python

```
from sklearn.ensemble import  
    RandomForestRegressor  
  
bosque = RandomForestRegressor(  
    n_estimators=200, random_state=42)  
bosque.fit(X_tr, y_tr)  
  
print(bosque.score(X_te, y_te))
```

>_ Output

0.980

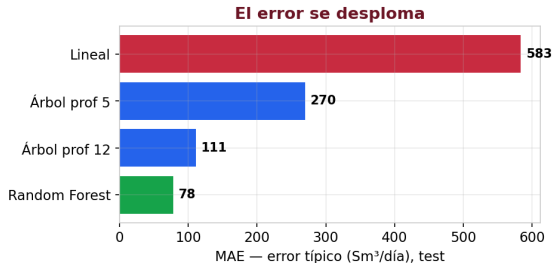
→ `n_estimators`: cuántos árboles. 200 es un buen punto de partida.

✓ $R^2 = 0.98$ en datos que nunca vio.

✓ Y sin estandarizar, sin transformar, sin decirle nada de la física.

El Marcador Final

Mismo problema, mismos datos de test. Solo cambió el modelo.



	R ²	MAE
Lineal	0.60	583
Árbol p5	0.89	270
Árbol p12	0.96	111
R. Forest	0.98	78

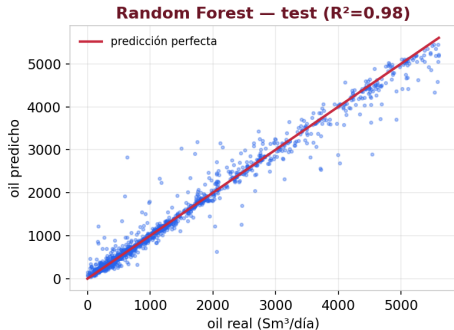
El error típico

bajó de **583** a **78** Sm³/día: **7.5 veces menos**.

Para un pozo de 1264 Sm³/día de media, es pasar de inutilizable a operativo.

El Veredicto Visual

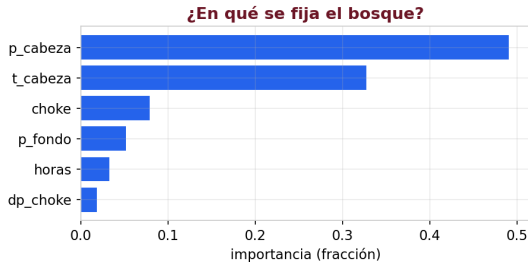
Predicho vs real en el test: el medidor virtual funciona.



- ✓ Los puntos se pegan a la diagonal en **todo** el rango: días de 200 y días de 5 000 Sm^3 .
- Recuerden la Clase 1: la recta del declive predecía **producción negativa**. El bosque no comete esos absurdos: nunca predice fuera del rango que vio.

¿En Qué se Fija el Bosque?

El bosque no da coeficientes, pero sí dice cuánto usó cada variable.



🏆 p_cabeza (49 %) y t_cabeza (33 %) dominan: la presión y temperatura de cabeza **son** la firma del caudal.

➔ Coincide con la física: lo que pasa por el choke se refleja en las condiciones de cabeza.

⚠ Ojo: importancia $\neq$ causalidad. Dice qué **usó** el modelo, no qué **causa** el flujo.

El Precio: Poder vs Explicabilidad

Nada es gratis. Lo que ganamos en precisión lo pagamos en transparencia.

	Lineal/Logística	Árbol	R. Forest
Captura curvas	no	sí	sí
Resiste overfitting	sí	no	sí
Se explica completo	sí (fórmula)	sí (diagrama)	no (200 árboles)
Necesita estandarizar	sí	no	no

- Del bosque solo podemos explicar las **importancias**, no el camino de cada predicción.
- ✓ Por eso la práctica profesional: **empezar simple** (línea base explicable) y **escalar** solo si el negocio necesita la precisión extra.

Segundo Problema: Clasificación

Volvemos a FORCE 2020 con una frontera no lineal

La Pregunta es la Misma; el Modelo Ahora Puede Doblar

Clase 2: con registros de pozo, decidir si una profundidad es arenisca o lutita.

- ▶ **Dataset 2:** FORCE 2020, 62 792 muestras de 11 pozos.
- ▶ **Features:** GR, RHOB, NPHI, DTC y RDEP.
- ▶ **Target:** arenisca = 1; lutita = 0.
- ▶ **Costo:** perder una arena cuesta 10 veces más que una falsa alarma.

Lo nuevo

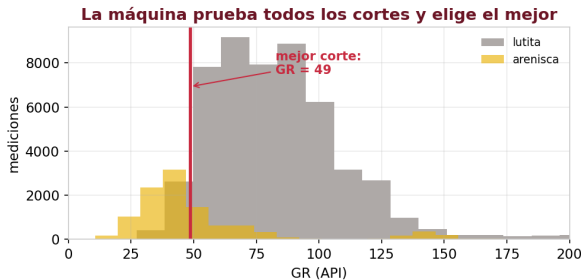
La logística busca una frontera plana. El árbol puede combinar reglas como:

GR < corte

y NPHI < corte

y DTC < corte

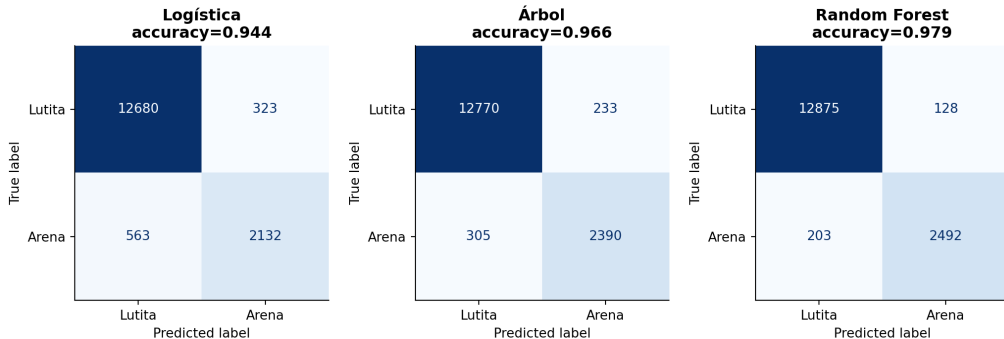
Un Solo Cutoff ya Tiene Sentido Geológico



- ▶ El árbol busca automáticamente un corte de GR.
- ▶ GR bajo concentra areniscas; GR alto, lutitas.
- ▶ La zona de traslape exige nuevas preguntas con NPHI, DTC y RHOB.

Eso es crecimiento recursivo: resolver primero la división más útil y después refinar cada lado.

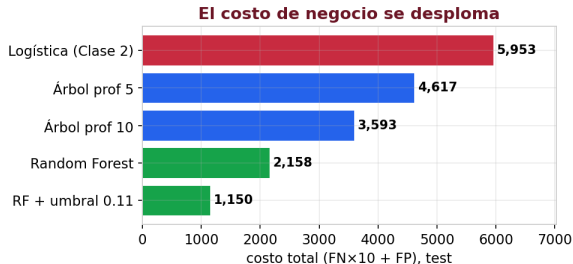
Tres Modelos, los Mismos Datos de Prueba



La matriz permite ver dónde mejora: falsos positivos y, especialmente, arenas perdidas (falsos negativos).

Accuracy sola no cuenta esa historia.

El Resultado Importante: el Costo Baja



- ▶ La logística de la Clase 2 sirve como baseline.
- ▶ El árbol captura interacciones y reduce arenas perdidas.
- ▶ El bosque baja el costo de 5 953 a 2 158: **64 % menos**.

Modelo mejor = mejor decisión bajo el costo definido, no simplemente mayor accuracy.

El Cambio en Código es Pequeño

Python

```
from sklearn.ensemble import  
    ↪ RandomForestClassifier  
  
rf = RandomForestClassifier(  
    n_estimators=200,  
    random_state=42)  
  
rf.fit(X_train, y_train)  
proba = rf.predict_proba(X_test)[:,  
    ↪ 1]
```

- ▶ Sigue entregando probabilidades.
- ▶ El umbral y la matriz de costos de la Clase 2 siguen vigentes.
- ▶ No necesita StandardScaler.
- ▶ Puede mostrar importancia de variables.

Cierre

Lo que se llevan hoy

Lo Que Aprendimos Hoy

Conceptos

- ✓ No linealidad: el paso deja de ser parejo
- ✓ La recta/logística fracasan (las lunas)
- ✓ Árbol = cascada de cutoffs **aprendidos**
- ✓ max_depth y el overfitting visible
- ✓ Random Forest: promedio de muchos árboles
- ✓ Importancia de variables ($\neq$ causalidad)
- ✓ El trade-off poder vs explicabilidad

Herramientas

- ✗ DecisionTreeRegressor / Classifier
- ✗ RandomForestRegressor / Classifier
- ✗ plot_tree, feature_importances_
- ✗ max_depth, n_estimators

La idea para llevarse:

la física de producción es curva. Cuando la recta se queda corta, los árboles la capturan — y el bosque la captura **sin memorizar**.

Lo Que Sigue

Ya tienen modelos lineales y no lineales. Falta afinar cómo los validamos y ajustamos.

- ➔ **Validación seria** — validación cruzada, y el detalle que dejamos anotado: evaluar con **pozos completos** por fuera.
- ➔ **Afinar las perillas** — búsqueda sistemática de hiperparámetros (`max_depth`, `n_estimators...`).
- ➔ **Multiclase** — las 11 litologías reales del FORCE 2020, con su matriz de penalización de la competencia.

La pregunta de siempre sigue mandando: ¿qué decisión habilita este modelo y cuánto cuesta equivocarse?

¡Gracias!

Carlos Enrique Mosquera Trujillo

✉ cmosquerat@unal.edu.co

Machine Learning for Petroleum Engineers Using Python

SLB Ecuador · UDLA · 2026